WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

Cangjie Magic 介绍

主讲人：陈冬杰



目录

- 示例演示
- 项目介绍

WORKSHOP 研讨/分享/感悟

快速上手 Cangjie Magic

仓颉 Cangjie

构建自动化 Agent

模型配置
接入三方/本地模型服务
可自定义扩展模型

规划方法
选择内置规划方法
可自定义扩展规划方法

提示词编写
支持提示词模式

流式输出

```
@agent[
  model: "deepseek:deepseek-chat",
  executor: "react",
  mcp: [
    { command: "node", args: ["${MARKDOWNIFY_DIR}/dist/index.js"] },
    { command: "docker", args: [ "run", "mcp/filesystem", "..."] }
  ]
]
class FileAssistant {
  @prompt[pattern: ERA] (
    expectation: "遵从指导逐步完成任务",
    role: "你是文件助手，帮助用户管理文件",
    action: "针对需求，逐步规划，你可以使用工具..."
  )
}
let agent = FileAssistant()
let asyncResp = agent.asyncChat(
    AgentRequest("帮我把所有 pdf 转换为 markdown 格式")
)
for (data in asyncResp) {
    print(data)
}
```

MCP 协议支持

仓颉 Cangjie

实现 RAG 问答应用

```
@agent[
  model: "deepseek:deepseek-chat",
  executor: "naive",
  rag: {
    source: "./docs/tutorial.md",
    mode: "static"
  }
]
class QABot {
  @prompt[pattern: ERA] (
    expectation: "代码块被标签 ```cangjie 和 ```包裹",
    role: "简单问答助手",
    action: "搜索文档获取知识并回答问题"
  )
}

main () {
  Config.defaultEmbeddingModel =
ModelManager.createEmbeddingModel("ollama:nomic-embed-text")

  let bot = QABot()
  println(bot.asyncChat("Agent RAG 怎么编写"))
}
```

RAG 配置
预置文件类型快速RAG接入
可自定义扩展RAG

模型配置
设置 embedding 模型

仓颉 Cangjie

开发 MCP 服务器

服务端

```
import magic.mcp.MCPServer

@tool[description: "Add two numbers"]
func add(a: Int64, b: Int64): Int64 {
  return a + b
}
@tool[description: "Multiply two numbers"]
func multiply(a: Int64, b: Int64): Int64 {
  return a * b
}

@agent[
  model: "deepseek:deepseek-chat",
  tools: [add, multiply]
]
class Calculator {
  @prompt(
    "精通小学课本上的算术计算"
    "对于加减乘除运算得心应手"
  )
}

main() {
  MCPServer.startWith([Calculator()])
}
```

工具定义
将 Cangjie 函数自动转换为工具

工具接入
自定义函数、Agent工具、内置工具等

启动 MCP Server

客户端

```
@agent[
  model: "deepseek:deepseek-chat",
  mcp: [
     { command: "calculator.exe", args: [] }
  ]
]
class HomeworkAssistant {
  @prompt(
    "你能够协助小学生们完成家庭作业“
    "让他们在各个学科均衡发展"
  )
}
```

仓颉 Cangjie

接入自定义模块

新模型

```
import magic.core.model.*

class EchoModel <: ChatModel {
  public func create(req: ChatRequest) {
    return ChatResponse(
      req.dialog, "echo-model"
    )
  }

  public func asyncCreate(req: ChatRequest) {
    throw UnsupportedException()
  }
}

let agent = FooAgent()
agent.model = EchoModel()
```

实现 ChatModel 接口

通过 model prop 设置

新规划方法

```
import magic.core.agent.*

class NothingExecutor <: AgentExecutor {
  public func run(req: AgentRequest) {
    return AgentResponse(
      "Do nothing"
    )
  }

  public func asyncRun(req: AgentRequest) {
    throw UnsupportedException()
  }
}

let agent = FooAgent()
agent.executor = NothingExecutor()
```

实现 AgentExecutor 接口

通过 executor prop 设置

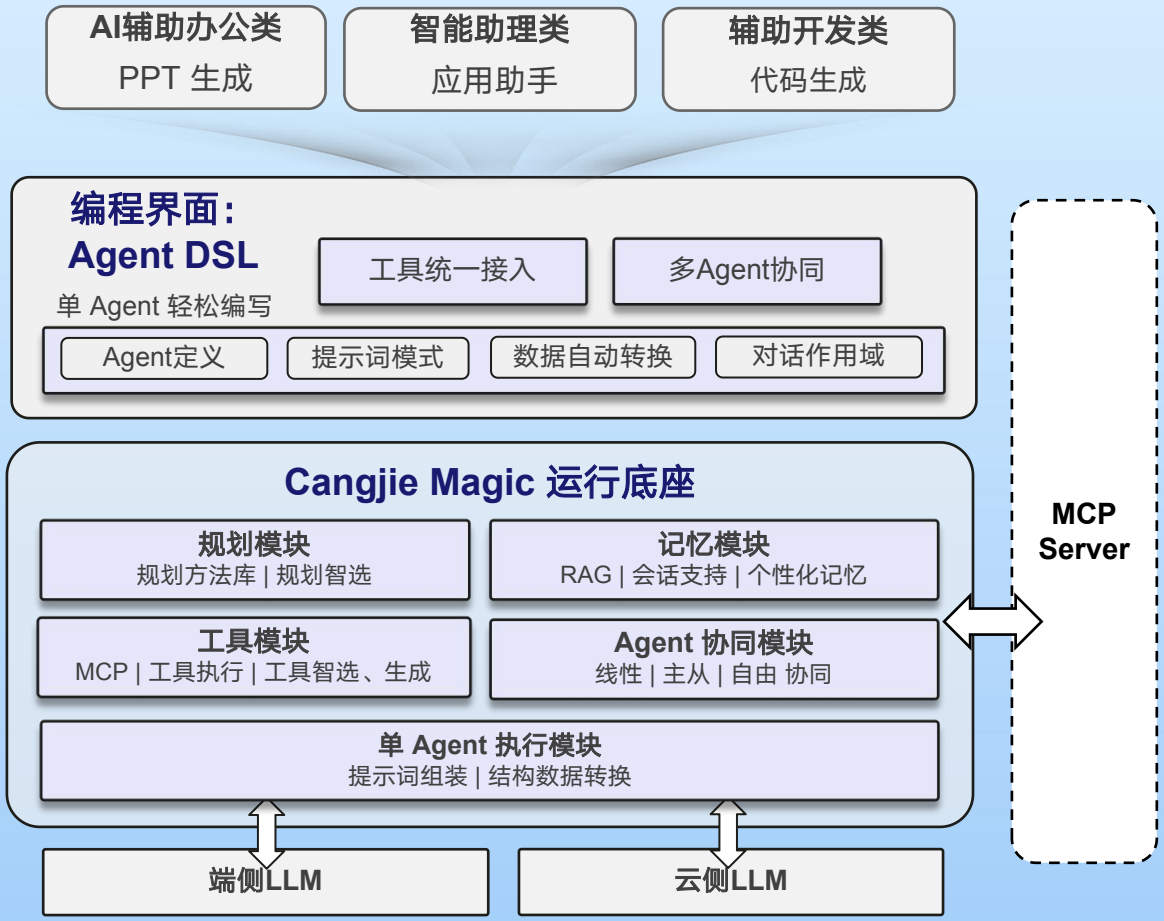
仓颉 Cangjie

WORKSHOP 研讨/分享/感悟

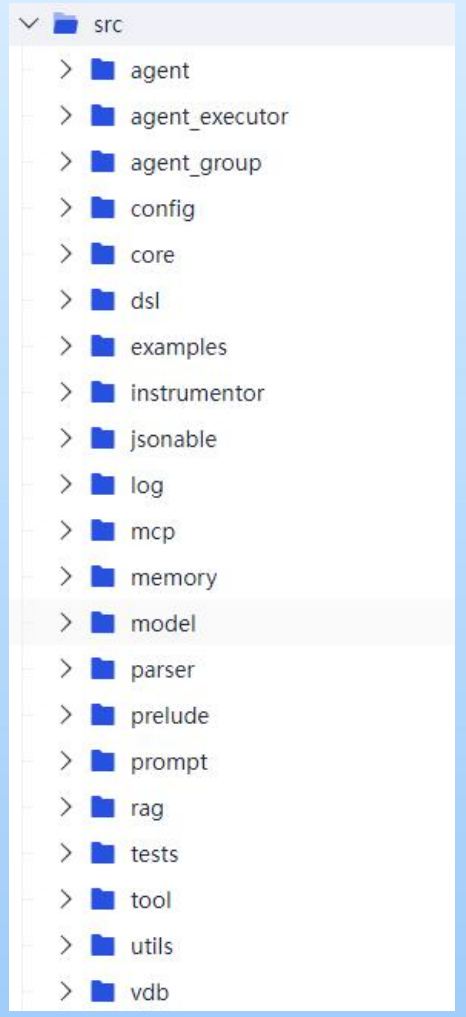
Cangjie Magic 项目介绍

仓颉 Cangjie

Cangjie Magic 有什么？



Cangjie Magic 代码结构



仓颉 Cangjie

使用 DSL 高效开发

通过 DSL 简化 Agent 编写和提供灵活的 Agent 交互方式

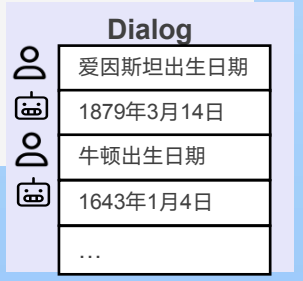
```
@agent[
  model: "deepseek:deepseek-chat",
  executor: "react"
]
class Foo {
   @prompt[pattern: ERP](
     expectation: "回答要求简要凝练",
     role: "你是问答助手",
     action: "若无法直接回答问题，则通过工具搜索并整理
答案"
   )
}
```

模型、规划等一键配置

预置模式引导提示词“完形填空“

维护对话历史，完成多轮交互

```
@dialog[agent: foo](
   "爱因斯坦出生日期" -> d1: MyDate
   "牛顿出生日期" -> d2: MyDate
   "谁的年龄更大" -> name: String
)
```



流式符号简化 agent 协同

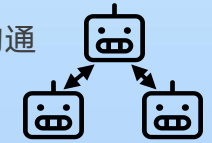
```
let linearGroup: LinearGroup = ag1 |> ag2 |> ag3
```

线性协同：Agent依次执行并传递消息



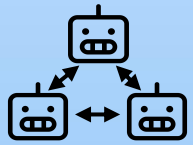
```
let leaderGroup: LeaderGroup = ag1 <= [ag2, ag3]
```

主从协同：主 Agent 选择成员 Agent 进行点对点沟通



```
let freeGroup: FreeGroup = ag1 | ag2 | ag3
```

自由协同：Agent 自由通信，消息广播



仓颉 Cangjie

统一接入工具

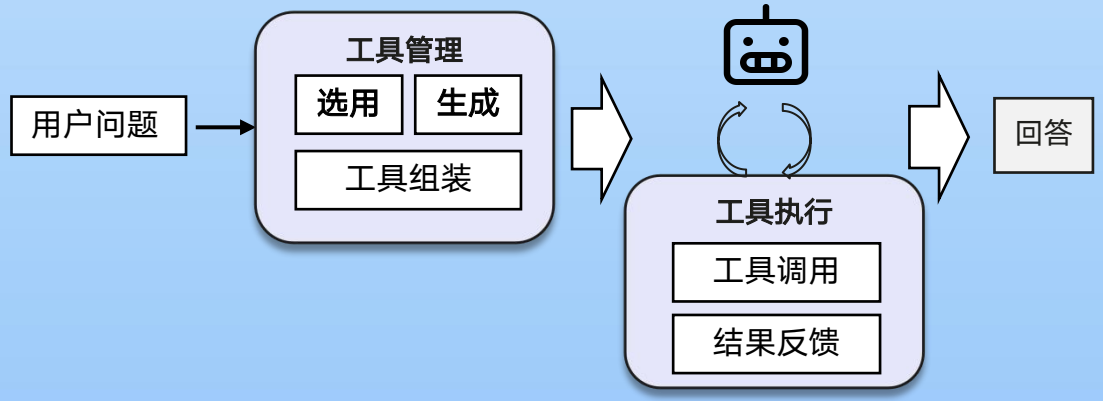
DSL 提供标准化配置化工具编写，支持标准化接入

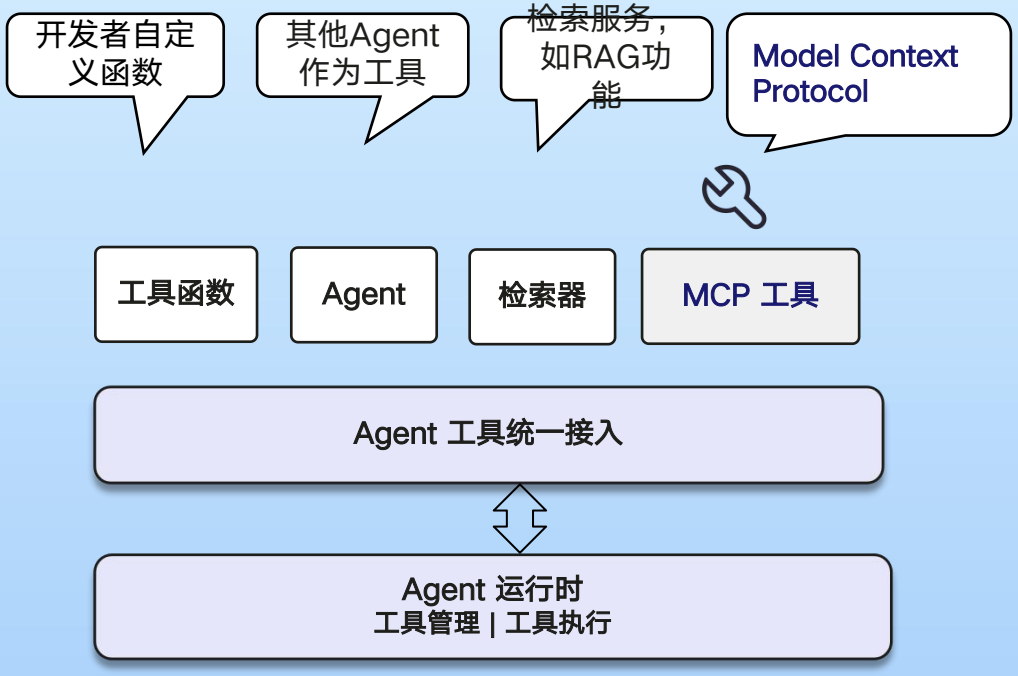
```
@tool[description: "根据问题搜索相关内容",
      parameters:{ q: "搜索问题" } ]
func search(q: String):String { ... }

@agent[
    tools: [search,
            BarAgent().asTool(),
            VectorDB().asRetriever()],
    mcp: [
      {name: “docker”, args: ["mcp/filesystem"]}
    ]
]
class Foo { ... }
```

基于自定义工具和MCP工具，让开发者构建的Agent有更强的执行能力

Agent 运行时智能选用工具并组装





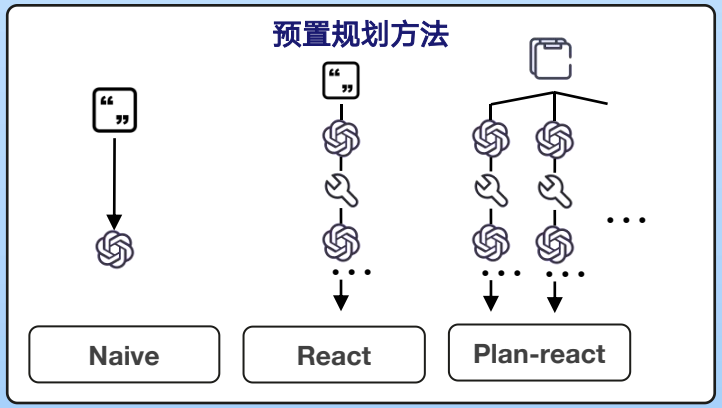
仓颉 Cangjie

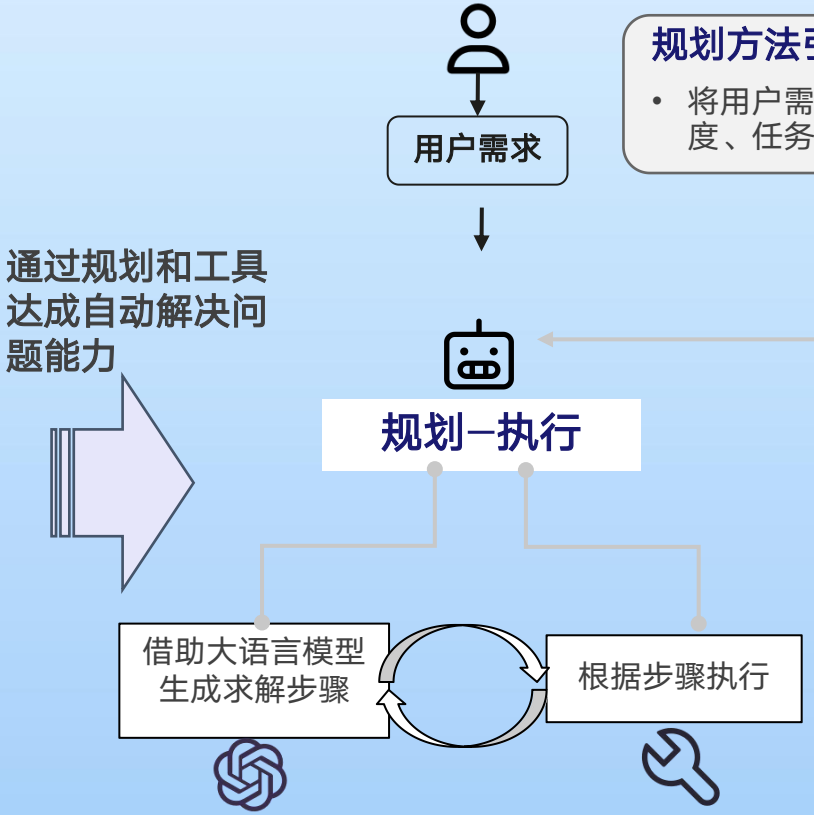
规划方法

DSL 提供两种规划方法编写

- 配置规划方法
- 通过DSL开发自定义新方法*

```
@agent[
  executor: "react"
]
class Foo { ... }
```





规划方法引导Agent完成问题求解

- 将用户需求拆解为意图识别、任务拆分与调度、任务执行、归纳整理等步骤，逐一执行

名称	说明	时长*	能力
Naïve方法	大语言模型直接生成结果	2s	
React方法	根据中间结果让大语言模型决策下一步骤	20s	
Plan-react	细粒度分解并使用 react执行	114s	
...	...		

规划方法有不同的求解能力和运行时开销

*时长由执行 Gaia 用例统计而来，采用 Deepseek 模型

仓颉 Cangjie

项目代码解读

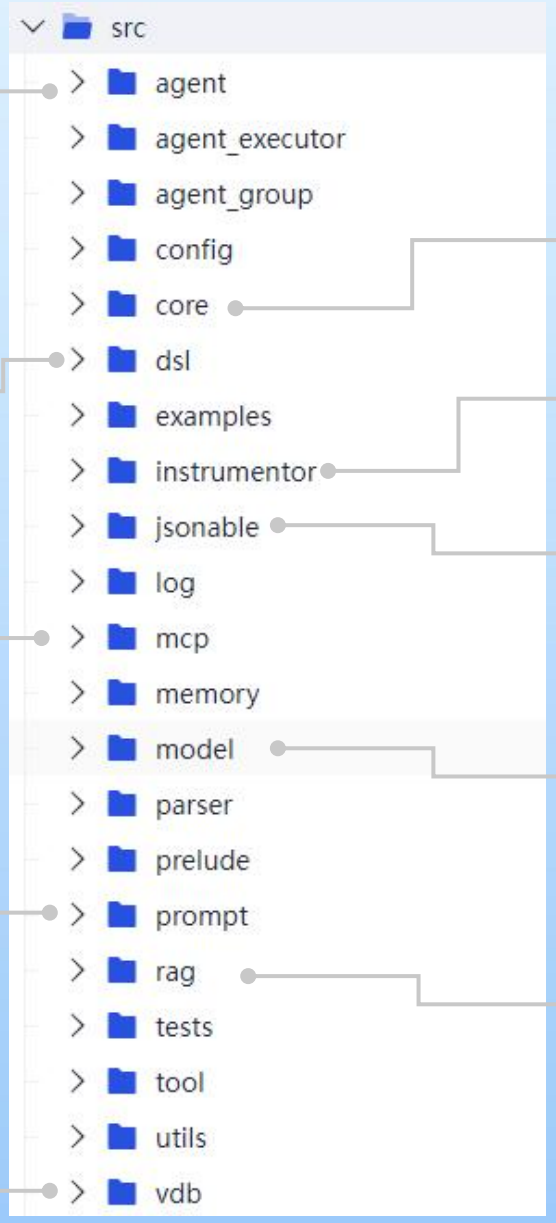
Agent 具体实现，包括 Agent 的基类、几种常见的 Agent

Agent DSL 实现，即各种宏（代码可能比较难理解）

实现 MCP 协议：主要 MCPClient 和 MCP Server 类型

提示词模式的具体实现

向量数据库实现



定义各种关键接口，如 Agent、ChatModel 等

预设的插桩点，但目前未使用

主要实现 Jsonable 接口，支持类型对 Json 的序列化/反序列化并获取 Json schema

对话模型、向量模型等具体实现

Retriever 具体实现

仓颉 Cangjie

QA

仓颉
Cangjie

WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

谢谢



